Onnxruntime

目录

- 图优化

图优化

onnxruntime通过 `TransformerLevel` 来控制优化级别，

</> C++ | 收起

```cpp
// Graph transformer level
// refer to docs/ONNX_Runtime_Graph_Optimizations.md for details
enum class TransformerLevel : int {
  Default = 0,  // required transformers only
  Level1,       // basic optimizations
  Level2,       // extended optimizations
  Level3,       // layout optimizations
  // The max level should always be same as the last level.
  MaxLevel = Level3
};
```

onnxruntime中定义了多组优化的基类，分别是： `GraphTransformer` 、 `RuleBasedGraphTransformer` 。

这两类区别不是很明显，此处分别举例查看其联系和区别；

首先看下 `GraphTransformer` 。

</> C++ | 收起

```cpp
class GraphTransformer {
// ...
  /** Apply the in-place transformation defined by this transformer to the provided Graph instance.
  @param[out] modified Set to true if the Graph was modified.
  @returns Status with success or error information.
  */
  common::Status Apply(Graph& graph, bool& modified, const logging::Logger& logger) const;
```

```
  /** Helper method to call ApplyImpl on any subgraphs in the Node. */
  common::Status Recurse(Node& node, bool& modified, int graph_level,
const logging::Logger& logger) const {
    int subgraph_level = ++graph_level;
    for (auto& entry : node.GetAttributeNameToMutableSubgraphMap()) {
      auto& subgraph = *entry.second;
      ORT_RETURN_IF_ERROR(ApplyImpl(subgraph, modified, subgraph_level,
logger));
    }
    return Status::OK();
  }

  virtual bool ShouldOnlyApplyOnce() const { return false; }

  virtual common::Status ApplyImpl(Graph& graph, bool& modified, int
graph_level, const logging::Logger& logger) const = 0;

// ...
}
```

GraphTransformer核心定义如上述代码所示，主要定义了接口规范，ORT针对控制流多subgraph结构，显示定义了 `Recurse` 接口，供pass开发者调用。以 `ConstantFolding` 常量折叠为例，查看 `GraphTransformer` 下如何写pass：

1. 首先是正常的初始化操作，必要的CHECK检查等；
2. 根据拓扑序进行for循环，对每个node进行处理；
    - `Recurse` 操作，如果有subgraph，则先处理subgraph；
    - 正常的constant fold逻辑 .....

`GraphTransformer` 可以认为适用于通用的图操作，图修改任务；

</> C++ | 收起

```
Status ConstantFolding::ApplyImpl(Graph& graph, bool& modified, int
graph_level, const logging::Logger& logger) const {
  // 初始化操作
  GraphViewer graph_viewer(graph);
  auto& order = graph_viewer.GetNodesInTopologicalOrder();
  for (NodeIndex i : order) {
    auto* node = graph.GetNode(i);
    // ...
    ORT_RETURN_IF_ERROR(Recurse(*node, modified, graph_level, logger));

    bool converted_to_constant = false;
```

```
    if (node->OpType().compare("Shape") == 0) {
      converted_to_constant = ConstantFoldShapeNode(graph, *node);
    } else {
      InitializedTensorSet constant_inputs;

      // Check if constant folding can be applied on this node.
      if (!graph_utils::IsSupportedProvider(*node, GetCompatibleExecutionProviders()) ||
          !optimizer_utils::IsOperationDeterministic(node->Domain(), node->OpType()) ||
          // constant folding does not support executing a node that includes subgraphs (control flow operators,
          // such as If/Loop/Scan, fall into this category). individual nodes in the subgraph will be processed
          // by the Recurse call above
          node->ContainsSubgraph() || !graph_utils::AllNodeInputsAreConstant(graph, *node, constant_inputs, excluded_initializers_)) {
        continue;
      }
      // ... 计算出常量，并存储进Proto中，省略
    }
    // 删除可折叠的计算节点
    if (converted_to_constant) {
      // Remove single-output node chain for inputs of the node
      auto p_ip_node = node->InputNodesBegin();
      const auto p_ip_node_end = node->InputNodesEnd();
      while (p_ip_node != p_ip_node_end) {
        const auto& input_node = *p_ip_node;
        // Update the node iterator before removing the corresponding node because removing
        // the node will invalidate the node iterator
        ++p_ip_node;
        graph_utils::RemoveNodesWithOneOutputBottomUp(graph, input_node);
      }

      // Remove the output edges of the constant node and then remove the node itself.
      graph_utils::RemoveNodeOutputEdges(graph, *node);
      graph.RemoveNode(node->Index());
      modified = true;
      have_updated_nodes = true;
    }
```

```
46
47   }
48 // ...
49 }
```

`RuleBasedGraphTransformer` 在 `GraphTransformer` 的基础上，与 `RewriteRule` 结合；先看下 `RewriteRule` ，其主要定义了以下接口，实现了语义：针对哪些算子（ `TargetOpTypes` ），在哪些情况下（ `SatisfyCondition` ），允许进行Rewrite（ `Apply` ）。

</> C++ | 收起

```
1  class RewriteRule {
2   public:
3    enum class RewriteRuleEffect : uint8_t {
4      kNone,                // The rewrite rule has not modified the
   graph.
5      kUpdatedCurrentNode,  // The rewrite rule updated (but did not
   remove) the node on which it was triggered.
6      kRemovedCurrentNode,  // The rewrite rule removed the node on which
   it was triggered.
7      kModifiedRestOfGraph  // The rewrite rule modified nodes other than
   the one it was triggered on.
8    };
9
10   /** Returns the node op types for which this rule will be triggered.
   If the op type of a node is not included in the
11       target op types of a rule, that rule would not be considered at
   all. Returning an empty list indicates that we
12       will attempt to trigger the rule for every op type. */
13   virtual std::vector<std::string> TargetOpTypes() const noexcept = 0;
14
15   /** Checks if the Node of the given Graph satisfies the conditions of
   this rule. The body of the rule will be
16       evaluated if this condition function returns true. This can
   include a more complex pattern matching (conditions
17       on the ascending or descending nodes of the node for which this
   rule was triggered) or some other properties
18       of the nodes. */
19   virtual bool SatisfyCondition(const Graph& graph, const Node& node,
   const logging::Logger& logger) const = 0;
20
21   virtual common::Status Apply(Graph& graph, Node& node,
   RewriteRuleEffect& rule_effect, const logging::Logger& logger) const =
   0;
22
```

```
  // ...
}
```

以常见的 `ConvBNFusion` 为例，

C++ | 收起

```
// in .h
/**
@Class ConvBNFusion

Rewrite rule that fuses two Conv+BN nodes to a single Conv node.

It is attempted to be triggered only on nodes with op type "Conv".
*/
class ConvBNFusion : public RewriteRule {
 public:
  ConvBNFusion() noexcept : RewriteRule("ConvBNFusion") {}

  std::vector<std::string> TargetOpTypes() const noexcept override {
    return {"Conv"};
  }

 private:
  bool SatisfyCondition(const Graph& graph, const Node& node, const logging::Logger& logger) const override;

  Status Apply(Graph& graph, Node& node, RewriteRuleEffect& rule_effect, const logging::Logger& logger) const override;
};
```

`SatisfyCondition` 的写法，类似于写了一个pattern， `Apply` 为真正的Rewrite过程，ConvBnFusion中在此函数内计算fuse后的weight和bias。

`RuleBasedGraphTransformer` 继承自 `GraphTransformer` ，主要实现了以下方法： `ApplyRulesOnNode` 针对节点，应用所有的rules；重写了基类的 `ApplyImpl` 方法，也是在一个拓扑序循环中依次调用 `ApplyRulesOnNode` 。

C++ | 收起

```
class RuleBasedGraphTransformer : public GraphTransformer {
  /** Registers a rewrite rule in this transformer. */
  Status Register(std::unique_ptr<RewriteRule> rule);

  common::Status ApplyRulesOnNode(Graph& graph, Node& node,
```

```
                                    gsl::span<const
std::reference_wrapper<const RewriteRule>> rules,
                                    RewriteRule::RewriteRuleEffect&
rule_effect, const logging::Logger& logger) const;

Status RuleBasedGraphTransformer::ApplyImpl(Graph& graph, bool&
modified, int graph_level, const logging::Logger& logger) const {
// ...
  for (NodeIndex i : order) {
// ...
    rules = GetRewriteRulesForOpType(node->OpType());
    if (rules) {
      ORT_RETURN_IF_ERROR(ApplyRulesOnNode(graph, *node, *rules,
rule_effect, logger));
    }
// ...
  }
}

// ...
}
```